

Revision Notes for Class 12 Computer Science Chapter 1

Exception Handling in Python

When you write and run Python programs, errors can happen either because of mistakes in the program's structure or due to unexpected situations during execution.

Python separates these problems into **syntax errors** and **exceptions**. Syntax errors occur when you don't follow the correct rules of Python—for example, if you forget a colon or type a keyword incorrectly. The Python interpreter identifies and shows these parsing errors before the program runs, stopping further execution until you fix them.

But, even if your code's syntax is correct, you may face other problems during runtime—these are known as **exceptions**.

An exception means something went wrong while the program was running. For instance, if your code tries to divide by zero or open a file that isn't there, Python raises an exception. These can cause your program to stop unless you handle them properly.

Common Built-In Exceptions in Python

Python comes with several built-in exceptions to help identify and handle common errors by name, making debugging easier. Here are some frequent exceptions and their reasons:

Name	Description
SyntaxError	Occurs when code is not written as per Python's rules.
ValueError	Raised when an operation gets an argument of correct type but inappropriate value.
IOError	Happens when a file cannot be opened.

KeyboardInterrupt	Raised if the user interrupts program execution (like pressing Ctrl+C).
ImportError	Triggered if the code tries to import a module which doesn't exist.
ZeroDivisionError	Occurs when you divide a number by zero.
IndexError	Happens if you try to access an invalid position in a list or sequence.
NameError	Raised if you refer to a variable that doesn't exist.
TypeError	Comes up when an operation or function is applied to the wrong data type.
OverflowError	Raised when a calculation exceeds the limits allowed.

Python also allows you to create your own exceptions for specific situations, called user-defined exceptions.

How Python Handles Exceptions

Whenever an error situation arises, Python creates an exception object that contains details about what went wrong and where. If you don't provide a way to handle the exception, Python will stop your program. Therefore, handling exceptions is crucial for robust programs.

To manage exceptions, Python uses special code structures: try, except, else, and finally blocks. The code that might cause an exception is put inside a try block. If any error occurs, Python jumps straight to a matching except block to handle it.

Basic Exception Handling Structure

Here's a typical pattern for handling exceptions:

```
try:  
    # code that might cause an error  
except ExceptionType:  
    # code to handle that error
```

You can have more than one except block to deal with different errors. If you don't specify any error type, the final except catches all exceptions not handled by earlier except blocks.

Multiple Except Blocks and the Else Clause

Sometimes, you expect more than one type of error from a piece of code. For example, if you're taking input and dividing numbers, errors could come from entering the wrong type of input or attempting a division by zero. You can use separate except blocks to handle these. Python also lets you use an else block, which runs only when no exception is raised in the try part.

```
try:  
    x = int(input("Enter a number: "))  
    y = int(input("Enter another number: "))  
    result = x / y  
except ValueError:  
    print("Please enter numbers only!")  
except ZeroDivisionError:  
    print("Cannot divide by zero!")  
else:  
    print("Result is:", result)
```

This arrangement helps in providing clear and useful feedback to users, making your program more user-friendly and reliable.

Using the Finally Clause

No matter what happens in a try block, you may want to do some final steps that must always occur (like closing files or cleaning up resources). The finally block is executed whether there was an error or not. It's placed at the end after the except and else clauses.

```
try:
    f = open("test.txt")
    # work with file
except IOError:
    print("Could not open the file.")
finally:
    print("Closing file or ending process.")
```

The code here ensures that the closing message is shown, even if opening the file fails.

Raising Exceptions With raise and assert

You're not limited to letting Python handle all exception detection. If you want to force an error (for example, to block unwanted user behavior), use the raise statement. You can raise built-in or your own exceptions and attach a message to help with debugging.

```
if denominator == 0:
    raise ZeroDivisionError("Denominator cannot be zero!")
```

The assert statement helps test whether a certain condition is true. If false, it raises an **AssertionError**. It's useful for quick checks while developing programs.

```
assert number >= 0, "Number should not be negative"
```

This will raise an exception and show the message if number is less than zero.

Why Exception Handling Matters

Organized exception handling builds reliable and easier-to-debug programs. It keeps your logic structure separate from error correction and makes error messages to users clearer. You can quickly pinpoint problems, guide the user to fix input mistakes, and avoid program crashes that lose all progress.

- Python distinguishes syntax errors (detected before running) and exceptions (detected while running).
- Many types of built-in exceptions exist, like `ValueError` and `IndexError`, to make error messages more meaningful.
- Use `try`, `except`, `else`, `finally` blocks for safe, clean code execution.
- The `raise` and `assert` statements give you control to handle or check for problems as needed.
- Catching exceptions keeps the program running instead of stopping at the first error.

In summary, mastering exception handling in Python means writing code that not only works but keeps working, no matter what data or situation it meets.